

Initial Experiments: Extending the MVP Simulator

William Dennis

June 2026

Extends the MVP simulator (`docs/mvp/mvp_specification.tex`) by enriching the state space, action space, and reward function, moving toward the full MDP defined in `docs/design/initial_design.tex`.

1 State Space Extension

References `docs/design/initial_design.tex` §3 and Issue #124.

1.1 Enriched StateView

1.2 Contextual Bandit Conditioning

1.3 Open Questions

2 Action Space Extension

References `docs/design/initial_design.tex` §3 and Issue #126.

2.1 ActionConfig

The action space is extended from 2 to 6 actions, each with a `burden_penalty` representing the cost of sending that type of message. The full set:

Action	Description	Burden Penalty
<code>no_message</code>	Do nothing	0.00
<code>progress_feedback</code>	Show step-count progress	0.10
<code>goal_reminder</code>	Remind of daily goal	0.15
<code>motivational_prompt</code>	Generic encouragement	0.20
<code>recovery_suggestion</code>	Suggest recovery activity	0.25
<code>walking_suggestion</code>	Suggest a walk	0.30

Table 1: The six actions and their burden penalties. Higher burden actions are expected to be more effective but risk user disengagement.

Burden penalties are stored as metadata in the config via `ActionConfig(name, burden_penalty)` but are not yet wired into the reward function. The `action_names` property on `MDPConfig` normalises either format (`list[str]` or `list[ActionConfig]`) to plain string lists for agent consumption.

2.2 Transition Model Updates

The transition matrix grows from $2 \times 2 \times 2$ to $6 \times 2 \times 2$ (6 actions, 2 states, 2 next states). Each action defines a row:

$$\begin{aligned} P_{\text{no_message}} &= \begin{bmatrix} 0.9 & 0.1 \\ 0.4 & 0.6 \end{bmatrix} & P_{\text{progress_feedback}} &= \begin{bmatrix} 0.75 & 0.25 \\ 0.3 & 0.7 \end{bmatrix} \\ P_{\text{goal_reminder}} &= \begin{bmatrix} 0.8 & 0.2 \\ 0.35 & 0.65 \end{bmatrix} & P_{\text{motivational_prompt}} &= \begin{bmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{bmatrix} \\ P_{\text{recovery_suggestion}} &= \begin{bmatrix} 0.65 & 0.35 \\ 0.25 & 0.75 \end{bmatrix} & P_{\text{walking_suggestion}} &= \begin{bmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{bmatrix} \end{aligned}$$

Columns are {sedentary, active}. **walking_suggestion** has the strongest effect in both states, making it the optimal action everywhere. Backward compatibility: **ActionConfig** dicts are accepted alongside plain strings via the updated schema validator.

2.3 Agent Compatibility

All four bandit agents (Thompson Sampling, ϵ -Greedy, UCB1, Decaying ϵ -Greedy) handle variable-length action sets without modification. The **action_names** property extracts plain strings from **ActionConfig** dicts, so agents continue to receive `list[str]`. No agent-side changes were required.

2.4 Open Questions

3 Reward Function Extension

References docs/design/initial_design.tex §3 and Issue #132.

3.1 Multi-Timescale Reward

3.2 Reward Composition

3.3 Synthetic Clinical Data

3.4 Open Questions

4 Combined Results

End-to-end evaluation on **rule_based_with_actions.yaml**: 50 random seeds, 450 timesteps each. Results use the best hyperparameters found per agent variant (see §4.2).

4.1 Optimal Reward

Under the 6-action MDP, the optimal policy always selects **walking_suggestion**, which maximises $P(\text{active})$ from both states:

$$P(\text{active} \mid \text{sedentary, walking}) = 0.4, \quad P(\text{active} \mid \text{active, walking}) = 0.8$$

Stationary distribution: $\pi_{\text{active}} = \frac{2}{3}$, $\pi_{\text{sedentary}} = \frac{1}{3}$. Expected per-step reward:

$$E[r] = 1.0 \cdot \frac{2}{3} + 0.0 \cdot \frac{1}{3} = \frac{2}{3} \approx 0.6667$$

Total optimal reward: $\frac{2}{3} \times 450 = 300$. Because the same action is optimal in both states, the contextual and non-contextual bounds coincide.

4.2 Hyperparameter Optimisation

Grid search (50 seeds, 450 steps, 316 configurations):

$$\begin{aligned} \text{TS: } & (\alpha_0, \beta_0) \in \{0.5, 1.0, 2.0, 5.0\}^2 \\ \text{EG: } & \epsilon \in [0.01, 0.50], \Delta 0.05 \\ \text{DEC: } & \epsilon_{\text{start}} \in [0.1, 1.0], \Delta 0.1; \quad T_{\text{decay}} \in [25, 700], \Delta 25 \\ \text{UCB: } & c \in [0.1, 10.0], \Delta 0.5 \end{aligned}$$

Agent (Standard)	Total Reward	Per Step	Last 50	vs Optimal
DEC ($\epsilon_{\text{start}} = 0.4, T_{\text{decay}} = 75$)	273.5 ± 31.5	0.608	0.638	−8.8%
UCB ($c = 0.5$)	270.4 ± 24.5	0.601	0.656	−9.9%
TS ($\alpha_0 = 0.5, \beta_0 = 5.0$)	267.7 ± 31.4	0.595	0.638	−10.8%
EG ($\epsilon = 0.1$)	263.7 ± 29.4	0.586	0.612	−12.1%
Agent (Contextual)				
DEC ($\epsilon_{\text{start}} = 1.0, T_{\text{decay}} = 150$)	263.3 ± 20.7	0.585	0.644	−12.2%
UCB ($c = 0.3$)	261.0 ± 20.0	0.580	0.620	−13.0%
TS ($\alpha_0 = 1.0, \beta_0 = 5.0$)	253.2 ± 24.2	0.563	0.610	−15.6%
EG ($\epsilon = 0.15$)	253.1 ± 21.1	0.562	0.609	−15.6%
Optimal ($\pi_{\text{active}} = \frac{2}{3}$)	300.0	0.667	0.667	—

Table 2: Best agent variants on the 6-action config (50 seeds, 450 timesteps). Contextual agents underperform their standard counterparts because $6 \text{ actions} \times 2 \text{ contexts} = 12 \text{ parameter sets per agent}$, dividing the data too thinly over 450 steps.

4.3 Learning Curves

Standard variants outperform contextual ones across all four algorithms, contrary to the 2-action MVP results. With 6 actions, the $\frac{2}{3}$ optimal bound is reachable without state conditioning (the same action is best in both states), so contextualisation adds data fragmentation without benefit. DEC’s ϵ decay provides effective exploration early while exploiting later, yielding the best overall result at 91.2% of optimal.

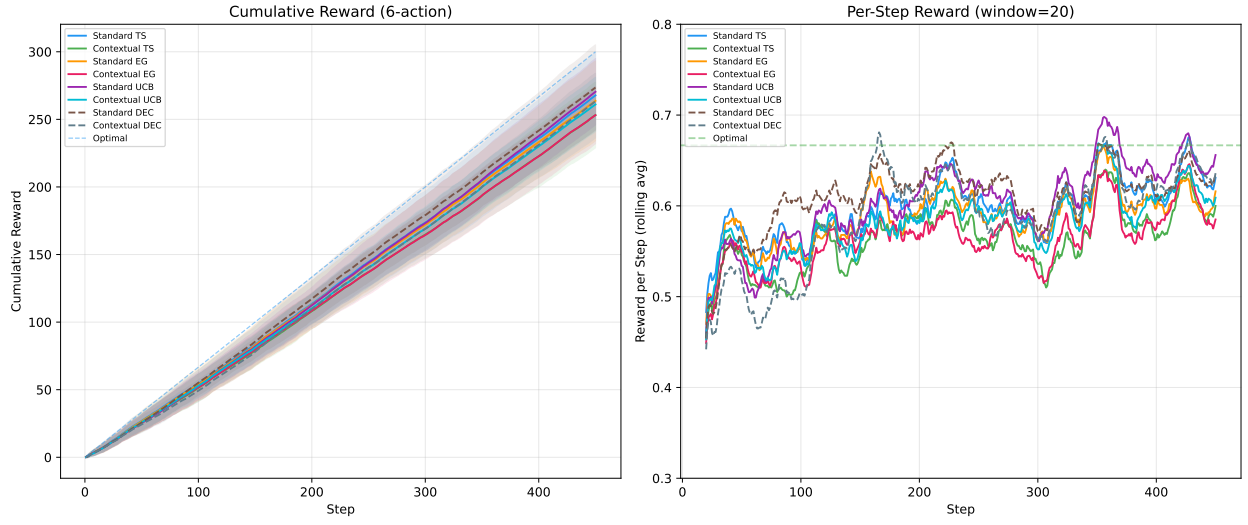


Figure 1: Cumulative and per-step reward (50 seeds, ± 1 SD). Standard DEC and UCB converge fastest; contextual variants lag due to data fragmentation across 12 context–action pairs.